

RESEARCH ARTICLE

WSC: A Crowd-Powered Framework for Mapping Decomposable Complex-Task With Worker-Set

Suneel Kumar¹  | Sarvesh Pandey² 

¹Department of Computer Science, Banaras Hindu University, Varanasi, India | ²Computer Science, MMV, Banaras Hindu University, Varanasi, India

Correspondence: Sarvesh Pandey (sarveshpandey@bhu.ac.in)

Received: 31 March 2024 | **Revised:** 3 September 2024 | **Accepted:** 23 September 2024

Funding: UGC Research Fellowship (Award Number: 220510027246).

Keywords: Aging technique | Complexity analysis | Crowdsourcing model | Dijkstra algorithm

ABSTRACT

The crowdsourcing platform serves as an intermediary managing the interaction between a requester who posts a decomposable task and a pool of workers who bid to solve it. Each worker intending to take up the task (partially or fully) decomposes it into multiple independent subtasks and submits it to the platform. Selection of a diverse set of workers (based on the bids received) to solve the decomposable task is challenging as it requires balancing factors like cost and quality while encouraging collaboration. We propose a Worker Set Computation (WSC) methodology to address these challenges by selecting a custom set of potential workers who can collaboratively complete the task with the optimal cost, in an efficient way. The aging technique is employed to dynamically update the weight of each worker, giving more weightage to the feedback received in the recent past. This, in turn, not only favors those workers who were rated well in the immediate past but also ensures that one odd feedback does not influence the overall rating heavily. We compare the performance of the proposed method against the state-of-the-art methods, considering the computational (and budget) requirements, as well as the aging-based worker rating.

1 | Introduction

Crowdsourcing is a distributed problem-solving methodology where a community of skilled individuals collaborates with a crowdsourcing platform to accept and solve tasks independently or collaboratively. The platform policies motivate people to participate in the problem-solving. In 2006, Jeff Howe, first coined the term “crowdsource” in an article in Wired, an American magazine [1]. Later, crowdsourcing has sought the attention of academia [2] and industries (Amazon Mechanical Turk (MTruk)) [3], Topcoder [4], Freelancer [5], Upwork [6], FlightFox [7], in the last few decades. Crowdsourcing is a modern distributed problem-solving paradigm that aims to gain services, gather materials, generate new concepts, and acquire other artifacts

from a comparatively large and open skilled community. Initially, crowdsourcing solves small-scale tasks such as blog creation, logo designing, and image tagging [8]. As its popularity increases, crowdsourcing can tackle much larger problems. With the expansion of crowdsourcing, it begins to address spatial crowdsourcing issues [9–11], where mobile workers can solve tasks like furniture installation and home improvement. Through crowdsourcing, a four-step model improves anti-pattern detection in the services [12]. Recently, a new crowdsourcing platform Papulii [13] was launched by Tech Mahindra [14]; allows collaboration between gig workers and esteemed organizations which demands human-in-the-loop services. Additionally, Populii provides services such as transcription, data collection, content rating, and annotation across various data formats. The task quality remains

Abbreviation: WSC, Worker Set Computations.

Suneel Kumar and Sarvesh Pandey contributed equally to this work.

high due to the competitive ecosystem within the crowdsourcing communities [1]. Since, there is a competition among the several crowdsourcing platforms, to design a mechanism to map the task with the workers is a calling task due to several issues. Numerous studies have been conducted to address this issue of crowdsourcing platforms and ensure the satisfaction of both customers and workers.

This paper presents a genuinely novel model for a crowdsourcing platform designed to solve complex decomposable tasks efficiently. Its practical application can benefit both academic communities and various industries by optimizing costs, scheduling tasks to reduce computational time, and enhancing overall efficiency [15, 16]. The workers on the crowdsourcing platform solve both simple and complex tasks at competitive prices due to the bidding process involved in cost declarations.

The workflow of the crowdsourcing platform is as follows: Requesters broadcast their tasks to the platform, which then releases these tasks to a pool of interested workers. Upon receiving the broadcasted tasks, workers decompose them into various subtasks as per their convenience, tagging each subtask with a cost. These subtasks and their associated costs are then returned to the platform. After that, the solution is returned to the requesters. The platform's challenge is to optimally assign tasks or sets of subtasks to workers and manage their execution effectively [17]. Several research studies have been done on the mechanism for assigning tasks to workers and managing crowdsourcing tasks. The feasible ideas management of the task in them is implemented and applied to the real-world practice [18, 19]. Despite the advancements and applications in this field, several issues and challenges remain in designing an effective crowdsourcing worker-task mapping mechanism [20–22]. For instance, non-transparency and unfairness remain two major issues in legacy crowdsourcing, particularly concerning incentive mechanisms, trust in worker profiles, and the worker's malicious activities. Han et al. [23] suggest an incentive mechanism for building the trust of the workers and requesters in crowdsourcing utilizing the smart contract core concept of Blockchain technology. Additionally, it provides the facility with a profile of workers working on the crowdsourcing platform (e.g., mTurk [3]), which can be trusted on another crowdsourcing platform (say upwork [6]). Blockchain technology has been used to improve free-riding and false reporting without the involvement of a third party [24]. Thus, a worker's skills can be utilized for more than two platforms, and more revenue can be generated concurrently by a single worker. However, the community focused on designing these mechanisms faces challenges related to space and time complexity due to the intricate nature of tasks and the large, skilled workforce competing to secure these tasks.

Our proposed model addresses critical issues and presents an innovative solution with the potential to resolve these challenges:

(C1). When the requester posts a task to the crowdsourcing platform, the pool of workers decomposes it into subtasks. Integrating all subtasks and their associated solution costs creates a weighted directed network supplemented with dummy uni-directional and bidirectional edges of zero weight. The Dijkstra Algorithm [25] is applied to this weighted directed network to determine the shortest path, which represents the total solution

cost, set of subtasks, and their associated workers of the entire task. The total path cost reflects the overall price of the solution for the requester's task. This approach effectively identifies the optimal set of workers to solve the task at the lowest cost [26], though challenges related to space and time complexity remain. We aim to improve the efficiency in two aspects: (1) Reduce the search space by pruning certain graph traversals using a key clue threshold. (2) Decrease the time complexity in selecting the appropriate set of workers, ensuring the optimal cost. The directed weighted network (see Figure 1) contains two unique nodes, a source, and a destination; by applying the existing method, we get the solution. Our focus is to reduce the traversal by avoiding extra nodes. We have a clue that we cannot find a feasible solution in that area where the decomposition of the task is less than a threshold value. Then, we cannot avoid unnecessary traversal of the entire network. We propose the WSC algorithm that performs better in terms of space and time complexity over the existing method, utilizing the clue of running some unnecessary nodes and edges. Our proposed WSC algorithm aims to bypass the highlighted red subgraph efficiently, and now WSC explores the remaining area of the graph (see Figure 1). The WSC explores the remaining network section, encompassing green and blue sections, and finds the shortest path (highlighted in green). In terms of space and time complexity, when compared to the current algorithm, the WSC performs better.

(C2). Our proposed algorithm determines a unique path; it may overlook multiple shortest paths with the same cost values. Consequently, the proposed method cannot effectively determine the optimal solution path among a set of equally viable paths. How can we handle this shortcoming? We propose an aging technique to address this limitation; we find a solution where the involved workers have a high reputation value, called aged weight. The static weight of the workers has been estimated with the academic scores of the students of a college who are supposed to be working for a crowdsourcing platform. The worker's dynamic weights are calculated based on the reviews received on the performance of their solved tasks.

(C3). To find the static weight of the workers, we have considered the metric undergraduate and post-graduate scores, post-graduate types (private, regular, or distance learning), and experience counted in years. The dynamic weights of the workers are updated in proportion to their performance. The aged weight of a worker is calculated by taking into account both the dynamic weights list earned throughout their work experience and static weights. The dynamic weight that the workers have earned is an integral component of their ongoing career experience. The aging technique is used to lead to fairness and honesty for the requesters and workers. The proposed aging technique has a core function called the decay function, which uses the decay rate. The function's decay rate establishes how weights' relative importance varies over time based on the list's chronological order.

1.1 | Real-Life Scenario

To illustrate the proposed idea, consider the following scenario: A user (requester) needs to digitize and transcribe their historical data. The he/she submits this request to the crowdsourcing platform, Upwork (say) [6]. The platform broadcasts this task to its

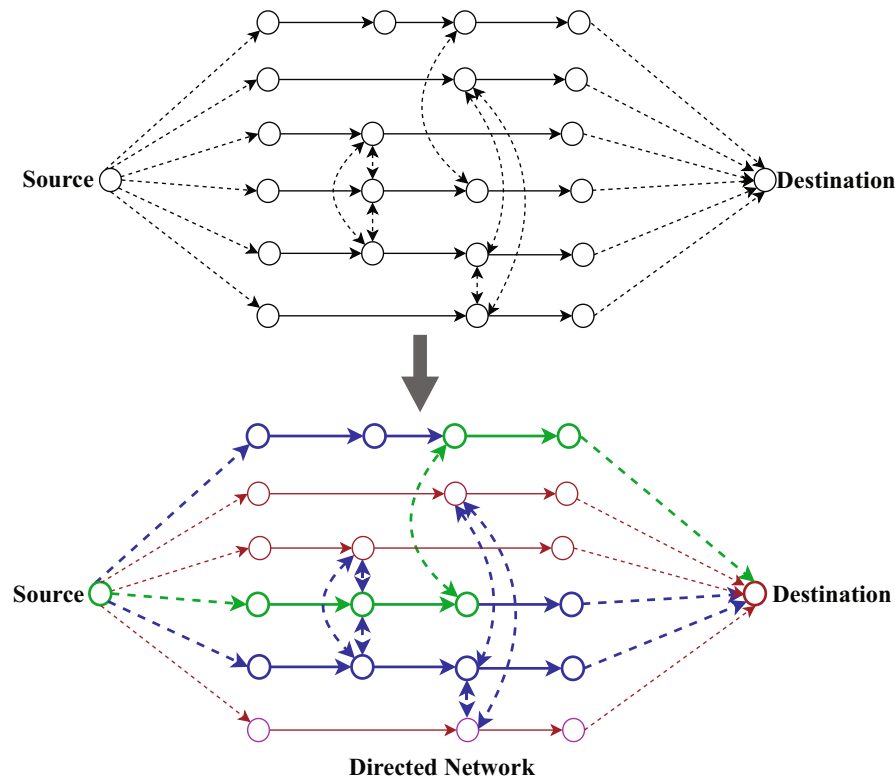


FIGURE 1 | The directed graph, the traversal (blue and green subgraph) for finding the shortest path (green), with the proposed WSC algorithm efficiently bypassing a subgraph (red).

TABLE 1 | Modules of the decomposable task considered in the real-life scenario.

Abbreviation	Description
S.D.	Scanning Documents
I.E.	Image Enhancement
O.C.R.	Optical Character Recognition
M.T.	Manual Transcription
T.F.	Text Formatting
M.G.	Metadata Generation
Q.A.	Quality Assurance

pool of workers, and five workers accept the task. Each worker then decomposes the task into smaller modular subtasks (see Table 1 for details); each worker decomposes the task according to their interest and convenience and bids a solution cost for each subtask (see Figure 2).

Our proposed method, WSC, first eliminates the workers' solutions, decomposing the task into some number of subtasks below a specified number called subtask threshold. In our scenario, the WSC eliminates the solutions of the subtasks submitted by worker W_2 and W_3 , because we set the four threshold values. Then, it selects a set of solutions from the remaining workers: W_1 , W_4 , and W_5 . The set of selected solutions completes the entire digitization and transcription task, with workers W_1 , W_4 , and W_5 chosen as the winners for that task (see the Task solution in Figure 2). The summation of the solution costs is optimal and very friendly to the requester (company). The estimated solution

cost of the entire process of digitization and transcription is $40 + 20 + 30 + 60 + 20 + 15 + 30 = 215$; which is the lowest cost among all the combinations of the subtask's solutions.

1.2 | Real-Life Example

On February 15, 2024, the Supreme Court of India (SCI) declared the Electoral Bond Scheme unconstitutional, stating that the lack of transparency in political party funding violates Indian citizens' right to information (RTI) under Article 19(1)(a) of the Indian Constitution. The Supreme Court of India directed the State Bank of India (SBI), the authorized bank for dealing with Electoral Bonds (EB), to provide the Election Commission of India (ECI) with relevant information, including the date of purchase, the name of the purchaser, the denomination of the Electoral Bond, and the name of the political party that redeemed the EB, by March 6, 2024. Moreover, SCI directed ECI to collect the relevant detailed information of EB and publish it to the ECI website [27] provided by the SBI.

First, we need to digitize and transcribe all the EB details on the ECI website to publish them. Completing the work within the specified deadline is challenging. Our proposed mechanism, implemented on the crowdsourcing platform, can efficiently solve this task.

1.3 | Major Contributions

The summary of our proposed contributions is given below:

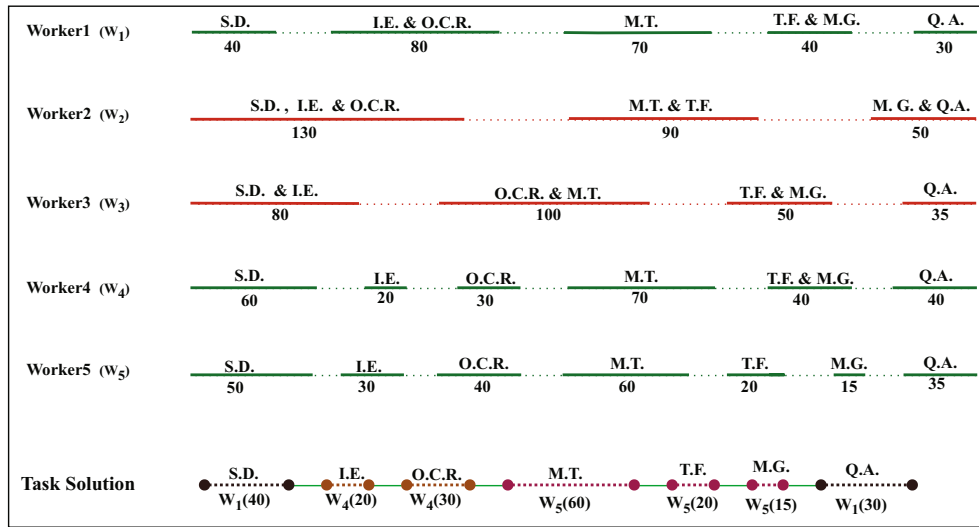


FIGURE 2 | Digitization and Transcription task and solution provided by the crowdsourcing platform.

1. We propose the WSC algorithm, which uses a threshold-based approach to select worker sets for solving decomposable tasks, significantly reducing space and time complexity by pruning unnecessary computations based on strategic clues.
2. Utilizing a dataset of college student records, assuming that these students are participants on a crowdsourcing platform, and estimating their static weight based on their numerical performance in college examinations for representing their initial profile numerical status. Further, introducing an innovative technique to calculate the worker's dynamic weights based on their dynamic performance using the feedback when they complete a task. The dynamic weight reflects the current status of the worker's reputation.
3. We propose an edge-cutting method called the aging technique to filter workers set for a solved task. The aging technique determines a numeric value for each solution quality; based on the solution quality, the aging technique is utilized as the tie-broken method when two or more solutions are similar.
4. The experiment's outcomes illustrate the benefits of our suggested model and the outperformance of our WSC algorithm against the outdated legacy approach while preserving mechanism trust.
5. We analyze the effect and performance of the aging technique in various previous distributions of the aged weights.

1.4 | Organization

The rest of the article is structured in the following way. The Section 2 discusses key works in similar and overlapping research areas. In the Section 3, we discuss the major problem. Next, in the Section 4, we focus on all the proposed methods and formulae used in finding insights. It lights briefly on the exciting work with the diagram. The Section 5 presents the setup environments where all the experiments are done. This section shows the performance results of our set objectives. In the Section 6, we

summarize our proposed works and briefly discuss the plan to extend further research.

2 | Related Work

This section surveys the works of literature on the mechanism of the crowdsourcing task management mechanism, its designing policies, requesters, and worker's interest in assigning the task.

Crowdsourcing is a distributed problem-solving method, it has reduced the teacher's workload in delivering the lectures and also a student may create course assignments by utilizing the proper tools and techniques. Crowdsourcing has affected learning in the computer science education system [28]. Different types of tasks—small, complex, decomposable, and non-decomposable—are solved by the crowdsourcing platform. A complex, decomposable task submitted to the platform can be particularly challenging to assign to the appropriate workers for optimal solutions. Various policies and task assignment mechanisms have been developed to address this issue. A network-based mechanism has been designed to assign workers to different subtasks of such decomposable tasks [26], and the assigned workers are declared winners. Initially, the requester's submitted task is divided into two parts. The optimal pair of solutions that minimizes the overall cost is selected, and the workers providing these solutions are declared the winners. It was later generalized to n -decompositions of the task, and the associated challenges were identified in the competitive crowdsourcing environment [29] and address the challenge of matching co-workers' skills for a decomposable task requiring diverse competencies [30].

Similarly, a large and complex task can be decomposed into independent multiple subtasks and efficiently solved through crowdsourcing. When the posted task cannot be decomposed into independent subtasks, such as a task of fiction writing, modifying one part of the story can affect the overall narrative. A technique for managing interdependent subtasks, drawing inspiration from expert story authors, revolves around revision

(a low-level goal) and reflection (a high-level goal). The technique is, in practice, implemented in Mechanical Novel on the mTurk crowdsourcing platform. Consequently, the Mechanical Novel produces stories of high quality utilizing this technique compared to the traditional crowdsourcing workflow [31].

The TM UNIFORM mechanism allocates heterogeneous tasks to various workers with different skill sets, ensuring budget feasibility and optimal utility. Further, introducing TM-RANDOMIZED improves the estimated guarantee. Simulating an experiment on the Wikipedia translation project provided practical feasibility guarantees with Mechanical Turk (MT) workers, supported by theoretical proof [32]. Due to its centralized nature, the existing crowdsourcing platform faces several significant issues, such as transparency in task and incentive distributions, reliability, and integrity of platform data, and the presence of malicious workers and payments. A blockchain-based framework, TSWCrowd, addresses these issues. This mechanism prioritizes assigning workers to high-priority tasks earlier. TSWCrowd increases the average worker's payment compared to the existing ABCrowd framework and significantly reduces total time compared to the VCG mechanism [33]. In addition to these issues, designing an effective incentive mechanism for sensing-based mobile phone applications and systems presents several challenges. Such a mechanism must encourage users and workers to participate and trust the crowdsourcing process. So, a Stackelberg game-based, platform-centric mechanism is designed in which the platform is in the central role, and the requesters are in the follower's roles. There is only one equilibrium for the Stackelberg game, and a practical computation technique has been created. Consequently, the utility of the platform is optimized, but it can't be further enhanced [34].

The primary purpose of the crowdsourcing platform is to satisfy the requesters and workers, and several features are implemented to fulfill these objectives. Matching the task with the workers according to their choice improves the quality of the solution task. To reach the goals of crowdsourcing, Gale-Shapley Matching (GSM) and SenseChain+ matching mechanisms based on the workers' preferences are proposed and integrated into the decentralized crowdsourcing platform [35].

Considering key features of crowdsourcing mechanisms, we have reviewed the literature and conducted comparisons based on the following themes: user collaboration, cost control, task-worker

assignment methods, worker trust mechanisms, and decentralized environments (see Table 2).

2.1 | Cost Control

The cost control issue of tasks in crowdsourcing environments is addressed in [23], while [37] discusses a pricing algorithm as an incentive mechanism. If the task acceptance rate by the workers exceeds expectations, the price is lowered, and the rate is decreased, the price is increased. The cost reduction is up to the 30% for a set deadline. The article [33] discusses task assignment and prioritizes assigning high-priority tasks to workers. Moreover, the payment is made automatically and decentralizedly using the smart contract. The associated smart contract is developed to improve the quality of the solution based on the suggested reward formula.

The worker not assigned with any task is paid zero amounts, and assigned workers for the task get the uniform and non-uniform amount [32]. For participation in the auction, the most appropriate pre-selected workers are paid a justifiable amount [36]. Due to the open nature of crowdsourcing, participation in the auction competition and acceptance of invitations is optional. The crowdsourcing platform manages payment after delivering the complete task solution to the requester and solicits feedback in the form of ratings. If the requester's budget is lower than the bids submitted by the workers, the platform rejects the request. It notifies the requester that their task cannot be allocated to any worker in that particular amount and conditions. In such situations, the requesters either need to extend the deadline or increase the budget and are required to re-submit the previous task to increase the task probability of acceptance by the crowd. By employing this method, we can efficiently monitor and regulate the cost parameter.

In our study, the WSC algorithm controls the solution cost. It is greedy about the optimal cost during the selection of the worker set. It calculates the summation of the cost of all subtasks, which combinations make the complete task. WSC controls the solution cost, minimizing the total cost of the entire task.

2.2 | Workers Collaboration

The proposed mechanism allows workers to decompose the task into subtasks and select the best combination of the subtasks to

TABLE 2 | Comparison of different studies.

Publications	Cost control	Workers collaborations	Worker-task mapping mechanism	Workers trust mechanisms	Decentralization
[23]	✓	×	✓	✓	✓
[26, 29]	×	✓	✓	×	×
[33]	✓	×	✓	✓	✓
[32]	✓	×	✓	×	×
[36]	✓	×	✓	✓	×
[34]	✓	✓	×	×	×
[11]	✓	✓	✓	✓	×
This study	✓	✓	✓	✓	✓

ensure that a requester can obtain the solution at an economical cost [26, 29]. A prototype for crowdsourcing utilizing envelope games (EG-CM) is designed for the determination of the task's actual price from workers. A notable feature is that workers only disclose the total cost, while the costs for individual subtasks remain undisclosed. If they seek to know the subtask's costs, they have to pay the fee to reveal the associated costs. By this means, collaboration can be done on a task for the optimization of the cost for the whole task. If the task's overall solution costs less than what the user has requested, the user accepts the solution, and the workers who contributed to solving the task are selected as winners. Further extended, the decomposition-solving problem strategy allows the workers to decompose the task into n parts instead of decomposing two parts. To solve the n -decomposable task problem, a network-based mechanism is designed to facilitate collaboration and deliver the solution at the optimal cost.

To facilitate collaboration, as described in the article [34], smartphone users act as workers to provide sensing services. Sensing the environment is a unit task; users connect their smartphones to the crowdsourcing platform to receive the assignment through the internet. All these facilities are provided via the cloud environment. A series of sensing tasks are released by the platform, and the smartphone users select sensing tasks per their devices' facilities and bid their desired payment. The platform hires the most suitable users to complete the tasks. As a result, a group of smartphone users work together with the platform to complete the tasks. In spatial crowdsourcing, tasks often require multiple workers. Leveraging a greedy approach and equilibrium principles, various algorithms have been proposed to assign tasks effectively and efficiently, aiming to maximize the overall rewards for workers [11].

In our insights, a set of decomposed subtasks of a posted task is selected from different workers, whose combination completes the task. Our WSC algorithm smartly selects subtasks from various workers, ensuring an optimal solution at minimal cost. This careful selection fosters effective collaboration, providing a comprehensive solution for the entire task.

2.3 | Worker-Task Mapping Mechanism

The task-worker mechanism is implemented through a smart contract tailored to the requester's requirements and quality control criteria. A worker selection mechanism is generated accordingly, while the crowdsourcing platform determines an incentive mechanism based on user needs and quality control standards. All the functions have been written in the smart contracts to deploy on the blockchain platform [23]. The workers can accept the task after making the required payment of some credit. A self-invoking template for worker selection locks the credit based on eligibility. If the worker is ineligible to solve the task, it is rejected. The process of rejecting and accepting workers is decided based on the majority agreement process.

To map a set of subtasks to a set of workers, which is binary decomposable tasks, is easy by making the binary combination of the subtask, which minimizes the total cost of the entire task [29]. Generalizing this concept for n decomposable task, the selection

of subtask and assignment to the workers is made by the Dijkstra algorithm so that the total overall cost can be optimized [26, 29].

The proposed TSWCrowd algorithm is designed to allocate tasks in a decentralized platform. Tasks are sorted based on their published time and are divided into two categories: partially allocated and fully allocated tasks. For each unallocated task, a pending workers set PW_i is determined. Subsequently, a group of workers is chosen to do the task. If a group of workers W_i has the capacity to execute a posted decomposable task to the crowdsourcing platform, then all the workers of that group are assigned to the task accordingly to ensure optimal task allocation [21].

Truthful Matching using Uniform Rate (TM-UNIFORM) is a task-worker assignment mechanism strategically allocating workers to tasks. This mechanism ensures budget feasibility, maintains incentive compatibility, and maximizes overall utility. Workers' compensation is determined by a fixed "buck-per-bang" rate r and their respective utility u , where each worker is paid an amount equivalent to $u \times r$. This rate r is consistent across all workers, ensuring uniformity in payments [32].

In the auction-based task-worker crowdsourcing assignment mechanism, workers first undergo an ability and qualification assessment to determine their suitability for executing the task. This initial phase helps to filter out unqualified participants and prevent auction flooding. In the subsequent phase, a limited number of qualified workers are selected [36]. These selected workers possess closely matching skills, making them eligible to participate in the auctions. Given the open nature of crowdsourcing, they can accept or decline auction invitations.

A single worker does not execute some tasks (furniture installation, home improvement, and decorations) due to the limited skills of that worker and need a set of workers to form a coalition for executing the entire task. So, in the spatial crowdsourcing (SC) for such a task, a Coalition-based Task Assignment (CTA) is proposed to handle such type of task [11]. To allocate high-value jobs to nearby workers, the first CTA adopts an acceptance probability and distributes tasks to them in a greedy manner. A priority-aware fairness mechanism is implemented and continuously refined to ensure fairness in the assignment process.

In our research, the WSC algorithm intelligently and efficiently assigns subtasks to a set of workers, ensuring cost optimization. Additionally, we have implemented a method that identifies the optimal solution path when the WSC encounters ambiguity in selecting the best task. After that, the requesters will receive the best solution according to their requested financial range.

2.4 | Worker Trust Weight Computation

A worker can accept a task by transferring a certain amount of credit, which is then locked by the worker selection function. If the worker has the skills to execute the provided task, the credit remains locked; otherwise, the worker is rejected. Utilizing the Zero-Knowledge Proof (ZKP) function, the requester can verify whether the task has been completed without revealing the details [23]. Thus, a user trusts the worker's eligibility due to the credit locking system.

Workers must register on the crowdsourcing platform to accept and receive tasks. While no formal trust measurement mechanism for workers is proposed, the decentralized nature of the platform inherently allows both the platform and requesters to trust in the workers' capacity and honesty [33]. An aptitude function that assigns a score between 0 and 1 is used to assess a worker's trust. A score of 1 indicates that the worker is fully qualified to solve the task, while a score of 0 signifies that the worker is unfit. A ranking function is also designed to rank workers, serving as a tie-breaker when two or more workers achieve the same aptitude score [36].

If workers perceive unfair treatment, they may lose trust in the SC platform and decline future participation. To address this, a marginal contribution reward distribution mechanism is in place. The priority-aware fairness concept is employed, prioritizing workers who are the first to be available on the SC platform. This approach fosters mutual trust, ensuring that users trust the workers' honesty, and workers, in turn, trust the platform [11].

In our approach, we build worker trust through a dynamic reputation profile. Among multiple sets of workers, one set is selected as the winner based on their superior dynamic weights, even when the overall task costs are identical for both sets for the same task. This mechanism ensures that both the requester and the platform can have confidence in the trust-building process.

2.5 | Decentralization

To address two major limitations of traditional crowdsourcing platforms, a decentralized framework powered by Ethereum has been designed [23]. This framework leverages smart contracts to implement key functions such as the selection process, incentive mechanisms, and task management. Smart contracts ensure transparency and fairness, building trust among requesters and workers. The platform enhances credibility by offering a transparent incentive mechanism, allowing workers to build a reliable profile that can be recognized across different platforms.

Blockchain-based Task Select Worker Crowd (TSWCrowd) system is designed to overcome several practical challenges of traditional crowdsourcing platforms [33]. TSWCrowd eliminates the need for third-party intermediaries by leveraging smart contracts within the Ethereum Blockchain framework, enhancing trust and transparency in task selection and incentive distributions.

2.6 | Comparison With Existing Work

A 2-way decomposable task on the crowdsourcing platform is solved using a cross-subtask selection technique that minimizes the overall cost by selecting the optimal pair of subtasks. To achieve the minimum-cost pair, the EG-CM is employed to reveal the true costs associated with each subtask [29].

When extending the concept from 2-way to n -way decomposable tasks, forming an optimal set of subtasks becomes more complex. To address this challenge, the Dijkstra algorithm is utilized to identify the best combination of subtasks and associated workers, ensuring that the total cost of completing the entire task is

minimized. Additionally, it has been observed that as the number of subtasks increases, the overall solution cost tends to decrease [26].

The problem with these solution scenarios is that they also involve workers who decompose the task into a smaller number of subtasks. This approach presents two key issues: First, it unnecessarily includes workers whose decomposition results in fewer subtasks, leading to a very low or negligible chance of winning. Second, this inclusion increases both space and computational complexity. In our research, we experimented 5.1 to support the argument that a feasible solution is more likely to exist in areas where the task is decomposed into a higher number of subtasks [26]. Building on previous work and experimental evidence, we propose a technique for selecting subtasks and their associated workers. In this approach, we discard workers who have decomposed the task into fewer subtasks than a pre-defined threshold. The threshold-based WSC implemented concept selects the workers as well as their respective subtasks and improves the space and time complexity (see Figures 10 and 12) over the other legacy mechanism [26, 29].

Moreover, to build the users' trust in the workers, the previous assessment method is either insufficient or cannot be directly applied to the workers to satisfy the users. To assess the requesters' trust in the workers, we have proposed a method that dynamically evaluates the trustworthiness of the workers. The technique updates workers' reputation weight regularly after the submission of the solution solved by them based on users' feedback of the solution. Furthermore, we analyze the method and how it improves workers' dynamic weight in different scenarios. We have resolved the issue by providing solutions in the budget range requested by users. We propose a technique to evaluate the quality of the solution that aids in delivering a better solution at minimal cost (see the Sections 4.9 and 4.10).

3 | Problem Statement

Assigning tasks to the most suitable workers who can complete them efficiently and earn money or reputation can be challenging. On the crowdsourcing platform, workers compete to complete tasks and earn rewards. Once the winners are announced, they receive payment from the shared budget provided by the requester through the platform. The primary objectives of the research work are to select an optimal set of workers, ensure cost efficiency, assign tasks with minimal computational effort, and deliver the highest quality solutions.

3.1 | Definition

A requester posts a task to the crowdsourcing platform, and a large community of sharp skills is associated with the platform, accepting and solving the task. The requester posts the task with a maximum budget (say B), which he/she is willing to pay to solve within a set deadline (say T). Consider, an n decomposable complex task is posted where there are m number of workers $W = \{W_1, W_2, W_3, \dots, W_m\}$ are working for the crowdsourcing platform. Assume the worker W_i decomposes the released task $S_i = \{(S_1^i S_2^i), (S_2^i S_3^i), (S_3^i S_4^i), \dots, (S_{k-1}^i S_k^i), (S_k^i S_{k+1}^i)\}$ where

$S_1^i, S_2^i, S_3^i, \dots, S_k^i$ are decomposed points. Also, consider a set of costs $C_i = \{c_1, c_2, \dots, c_k\}$ corresponding to each subtask decomposed by the worker W_i ; where c_p depicts the solution cost of each the subtask $(S_p^i, S_{p+1}^i) \in S_i$ provided that k_i^s are not required to be in the same sequence (see Figure 3).

The objective is to select a set of subtasks that combination complete the task and to group the associated workers who solve them at the optimal cost. Moreover, we aim to analyze the complexity to reduce it and compare the proposed mechanism with the existing method.

3.2 | Some Other Definitions and Theorems

- **Budget:** The maximum amount of money a requester is willing to pay to complete a task is called the budget. If the budget is less than all the bidding values posted by the workers, the requester either needs to increase the budget or compromise the deadline to get the task completed [26].
- **Theorem 1:** The directed task graph created by all the decomposed subtasks of the workers is an acyclic, and it does not have any self-loops [26].
- **Theorem 2:** The budget, F' , that is obtained utilizing the recent method is regularly less than or equal to the initial budget F [26].

4 | Methodology

This section presents a crowdsourcing model with a detailed workflow and introduces WSC. This optimal crowdsourcing mechanism efficiently maps decomposable tasks to workers, delivering better solutions than state-of-the-art methods.

4.1 | Previous Work

The problem is reformulated as a shortest path problem in a weighted directed network (WDN) to address the research challenge of decomposing tasks in crowdsourcing. A requester posts

the task on the crowdsourcing platform, and a set of workers is assigned to solve it. Based on their desire to finish each subtask, the workers divide the received task into several smaller tasks and apply a fee to each subtask. Combining all the subtasks from all the workers generates a WDN where the decomposed points work as nodes, the subtasks represent the edges, and the assigned costs of the corresponding subtasks define the edge weights. A source, a destination node, and some dummy edges with null weights are added to this network to make it a desirable WDN. On this WDN, the Dijkstra Algorithm is applied to determine the shortest path; the edges of the shortest path describe the selected subtasks, and the workers associated with these subtasks are declared winners. Moreover, the shortest path cost is the solution cost of the whole task. The following are the steps to construct the WDN and finding the shortest path.

1. Considering the breakpoints of decomposable tasks as a node (S_j^i) and each subtask as an edge (S_j^i, S_{j+1}^i) . The weight of each edge is the cost c_j of the corresponding subtask.
2. Bi-directional edges with null weights are added between the two equivalent breakpoints. Adding such types of edges is to allow workers to collaborate.
3. A source (S) node a destination node (D) dummy nodes are added to the creating network. The node S is to start the traversal, and D is the node where the traversal ends.
4. A set of uni-directional edges of null weights starting from S and to S_1^i , where $i = 1, 2, 3, \dots, m$ and again a set of uni-directional edges of null weights from ending nodes (here $S_4^1, S_3^2, S_3^3, S_4^4, S_4^5, S_3^6$) to the destination D are added to make the WDN.
5. Now, the WDN $G(V, E)$ is ready, allowing the algorithms to be applied to select the set of workers, determine the shortest path, and calculate the total cost of the solution for the entire task (see Figure 4).

We have considered that the requester has a fixed budget of $F = 9$ unit costs, the task solution is selected, and the involved workers are W_4 and W_1 . The selected subtasks are $(S_1^4, S_2^4), (S_2^4, S_3^4), (S_3^4, S_4^4)$ that combination complete the whole

W1	S_1^1	3	S_2^1	4	S_3^1	2	S_4^1
W2	S_1^2	6			S_2^2	4	S_3^2
W3	S_1^3	3	S_2^3	6		S_3^3	
W4	S_1^4	2	S_2^4	3	S_3^4	3	S_4^4
W5	S_1^5	3	S_2^5	2	S_3^5	5	S_4^5
W6	S_1^6	5		S_2^6	4	S_3^6	

FIGURE 3 | Subtasks decomposed by crowd workers and their associated solution cost.

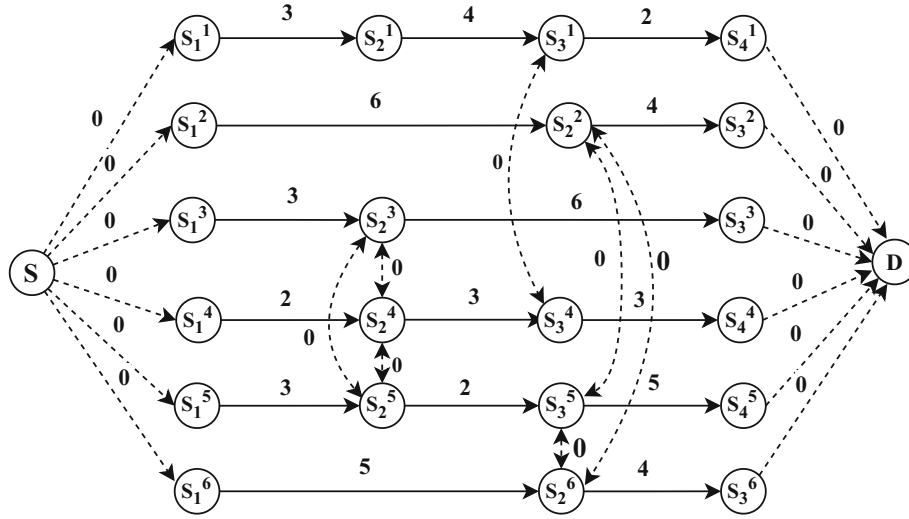


FIGURE 4 | The constructed weighted directed network.

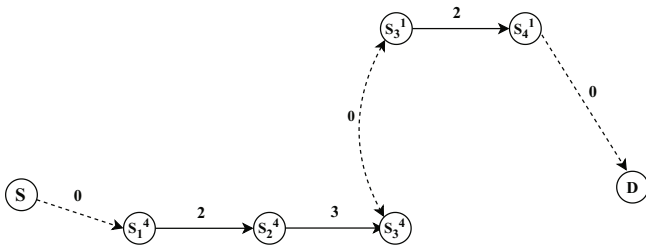


FIGURE 5 | Task solution as the shortest path by existing algorithm.

task, and cost of its solution is 7 which is fewer than the requested budget F . So, the requester saves 1 unit of cost using the crowdsourcing platform's leverage (see Figure 5).

4.2 | Proposed Work

The crowdsourcing platform's pool of workers breaks down the requester's posted task into several smaller subtasks. On increasing the number of subtasks decomposed by the workers in comparison with the former decomposed counting, the requester can acquire a more optimal solution by making a lesser payment, which encourages the crowdsourcing platform workers to break down the task into a more significant number of subtasks to be declared as the winners [26].

We have simulated experiments where workers decompose the task into more subtasks. The result analysis shows that feasible solutions are more likely to exist when tasks are broken down into more subtasks. In our proposed experiments, the workers decomposed tasks into a random number of subtasks. We employ a network-based greedy method to select the appropriate subtasks and their corresponding workers to solve them. The method creates a network first, then applies the algorithm to select the workers set and estimate the total solution cost. However, size of pool of workers increases, the generated WDN becomes more complex and time-intensive computational operations. Hence, to address these complexity-related concerns, either space or time or both, we provide a threshold-based idea to optimize the complexity. To resolve this issue, we assign a rank to the workers

associated with the platform. Thus, to address these complexity problems (in terms of time, space, or both), we assign a score to each worker in the pool according to how many subtasks they can decompose the task.

The recommended approach for workers ranking is given by Equation (10), which may be utilized to reduce the complexity. We only take into account workers who have decomposed the task into a number of subtasks that surpass the threshold T^θ to put our notion into practice. Alternatively, we can say that only workers whose ranks are greater than the pre-established threshold rank (R_θ) are considered. The variables R_θ and T^θ are interchangeable in this case. We also examine our method's relative performance improvement using graph representation. When a task has multiple solutions (i.e., different sets of workers) with the same cost, how will you determine the best solution for the user? Later, when two or more solutions from the pool of workers are available at the same cost, our goal is to resolve such challenges of selecting the best solution. Based on the worker's prior dynamic and static weights, we have employed an aging approach that assigns the worker an aged weight to address this issue. The most suitable solution will be selected from various alternatives, considering the aged-weight of the workers contributing to the solutions.

4.3 | Crowdsourcing System Model

This section outlines the crowdsourcing platform model and its components, detailing the flow of information and control (see Figure 6).

1. **Requesters:** The individual who has a gadget access the crowdsourcing website. He or she posts the decomposable task to the crowdsourcing website.
2. **Crowdsourcing Platform:** The digital platform consists of key components: a browser interface, a task management system, a crowdsourcing task assignment mechanism, and a payment system. The platform acts as an intermediary

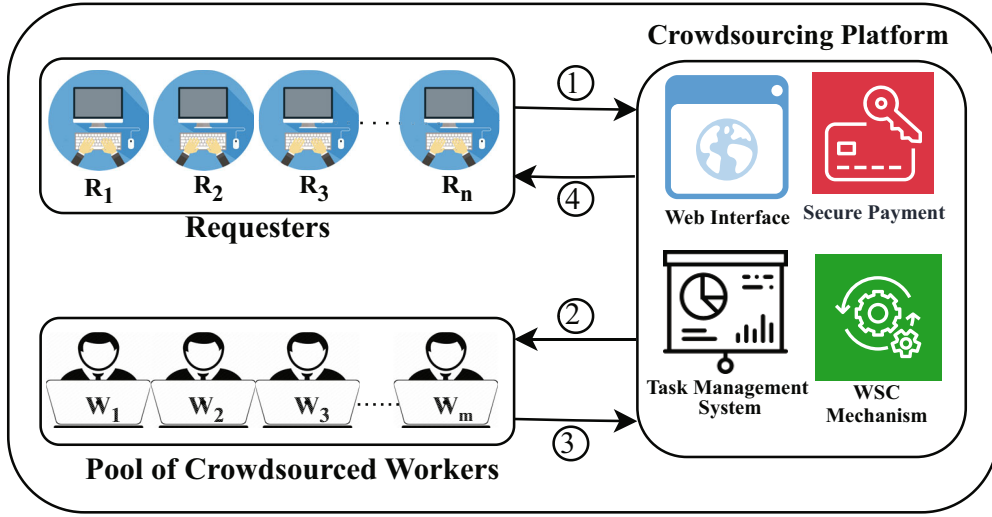


FIGURE 6 | Crowdsourcing model, its major components and information flow.

and manages the communication between workers and requesters.

3. **Pool of Workers:** It is a group of skilled persons who are eager to solve crowdsourcing tasks posted to the platform.

4.4 | Model Information Flow

The requester submits a decomposable task, which is then assigned to a pool of workers. The solution is returned to the requester in an optimal and cost-efficient manner, requiring minimal computational effort (see working flow in Figure 6).

1. Through the online interface, the registered requester submits their decomposable task to the crowdsourcing platform.
2. When the task is received from the requester, the crowdsourcing task management system performs the necessary pre-processing operations. The task managing system broadcasts the task to the pool of workers, and the interested workers accept the task and solve it.
3. Each worker submits their solution for the completed task, including the corresponding costs for each decomposed subtask, to the crowdsourcing platform. The crowdsourcing mechanism (WSC) selects the optimal combination of solutions submitted by workers for each subtask, ensuring the completion of the entire project.
4. Finally, the optimal solution to the task, with the lowest possible cost, is returned to the requester.

4.5 | Theoretical Model of the Proposed Methodology

Consider ξ as the set of all edges in the constructed weighted directed network (see Equation (1) for the edge set and Equation (5) for edge counting). Let ξ_i denote the set of edges for each component i (refer to Equations (2) and (6) for details). Let

V represents the total set of nodes (see Equation (3) for the node set and Equation (7) for counting nodes). Finally, let C be the set of all sub-costs associated with each subtask (see Equation (4), see Table 3 for used symbol descriptions).

$$\xi = \{(S, S_1^i) : \forall i = 1, 2, \dots, m\} \cup \bigcup_{i=1}^m \xi_i \cup \{(S_k^i, D)\} : \forall i = 1, 2, \dots, m, k = 1, 2, \dots\} \quad (1)$$

$$\xi_i = \{(S_j^i, S_{j+1}^i) : \forall i = 1, 2, \dots, m, j = 1, 2, \dots, k_i\} \quad (2)$$

$$V = \{S_0, D\} \cup \{S_j^i : \forall i = 1, 2, \dots, m \setminus j = 1, 2, \dots, k_i\} \quad (3)$$

$$C = \{C_{k_i}^i \in R^+ : \forall i = 1, 2, 3 \dots m, j = 1, 2k_i, \} \quad (4)$$

where R^+ represents the set of positive real numbers

$$n(\xi) = c \cdot 2 + \sum_{i=1}^m k_i + 2 \quad (5)$$

$$n(\xi_i) = k_i + 2 \quad (6)$$

A component represents a subgraph of the entire network, including S and D nodes.

The order of the network:

$$n(V) = \sum_{i=1}^m k_i + 2 \quad (7)$$

The complexity of the existing algorithm is as follows (see Equation (8)):

$$O(n(\xi)) = \log n(V) \quad (8)$$

The complexity of the core function of the WSC is given below (see Equation (9))

$$O(n(\xi')) = \log n(V') \quad (9)$$

where $n(\xi') < n(\xi)$ and $n(V') < n(V)$.

TABLE 3 | Notations.

Symbols	Descriptions
τ	Task posted by requester
m	Total workers in the pool
(S_j^i, S_{j+1}^i)	Subtask
C	Subtask costs set
c_j	subtask cost (S_j^i, S_{j+1}^i)
S	Source node of WDN
D	Destination node in WDN
c	count of connections between the equivalent breakpoints WDN
ξ_i	Set of edges made by the all subtasks decomposed by the worker W_i
$n(\xi_i)$	Number of edges in the component i including (S, S_1^i) and $(S_{k_i}^i, D)$
ξ	WDN's edge set
V	WDN's vertices set
ξ'	WSC's traversed edges set
V'	WSC algorithm's traversed vertices set
$n(\xi)$	Total edges count in the WDN
$n(V)$	Total vertices count in the WDN
$n(\xi')$	Total edges in ξ'
$n(V')$	Total vertices in V'
R_i	Worker W_i 's rank

We have reduced the traversal of the nodes and edges by the proposed algorithm WSC over the existing algorithm (see Equations (8) and (9)).

We have discussed the theoretical concepts and their experimental analysis. Our findings indicate that feasible solutions are more likely to be found in scenarios where the number of subtasks is significantly higher. Therefore, we select the subtasks from workers who have decomposed the task into more subtasks than a threshold. This means we focus on only the components where the number of edges exceeds a specified threshold. To ensure quality, we need to choose the threshold wisely, and to implement this idea in a real experiment, we provide a rank to each worker, which corresponds to the rank of the component, as follows (see Equation (10)):

$$R_i = \frac{\text{Number of the edges in the component } i}{\text{Total number of the edges in the network}} \times 100 = \frac{n(\xi_i)}{n(\xi)} \times 100 \quad (10)$$

If R_i and R_j denote the ranks of workers i and j respectively, where $R_i > R_j$, then worker i has decomposed the task into a greater number of subtasks than worker j .

We set a threshold T^θ to determine the shortest path by minimizing network traversal. The algorithm restricts traversal to only those subnetworks where the component ranking exceeds this threshold.

The number of traversal edges on imposing the threshold conditions (see Equation (11)):

$$\xi' = 2 \times c' + \sum_{i=1 \text{ AND } R_i}^m k_i + 1 \quad (11)$$

c' depicts the connection count between pairs of identical breakpoints when threshold conditions are applied. The number of vertices in the traversal network when a threshold condition is applied (see the Equation (12)).

$$n(V') = \sum_{i=1 \text{ AND } R_i}^m k_i + 2 \quad (12)$$

Here, the Boolean operator AND allows the summation of the components whose ranks exceed the threshold.

4.6 | WSC Concept

The proposed WSC method aims to identify a group of workers who can efficiently complete a task posted on the crowdsourcing platform to minimize total cost and with less exploring search space (see Figure 7). Utilizing the threshold concept, WSC smartly selects the set of workers whose cost summations of their subtasks are minimized. The major objective of the work using the proposed algorithm is to provide the desired solution at the optimal cost within the set deadline.

- **Worker-Set:** It is a group of workers who are declared task winners posted on the crowdsourcing platform.
- **Equivalent Break Points:** A pair of breakpoints that describe their subtask before or after is the same as in its functionality and output.
- **Workers Rank:** All the workers are associated with ranks based on the number of subtasks they have decomposed. Workers who decompose a higher number of subtasks receive a higher rank.

WSC is implemented to create a network and to find the shortest path. Algorithm 1 shows the pseudocode for creating the network, and Algorithm 2 describes the process of the worker set selection and optimized cost in the minimum effort. The process starts from Algorithm 1, the variable m is initialized with the number of workers in the input, and it is passed as a parameter (line 1, Algorithm 1).

For each worker W_i , an edge set ξ_i is generated to store all the edges connected to the subtasks decomposed by that worker. The weight of the edges is assigned to the worker's cost of the subtasks bid. The set ξ is utilized to store the edges corresponding to each subtask decomposed by the worker in which the edges associated with the source S and destination D dummy vertices are also included. Graph G is expanded by adding all of these edges created by each worker (see lines 2–15 in Algorithm 1). When examining two breakpoints from different workers, if they are found to be equivalent, bidirectional edges with null weights are added between them (see lines 16–17 of Algorithm 1).

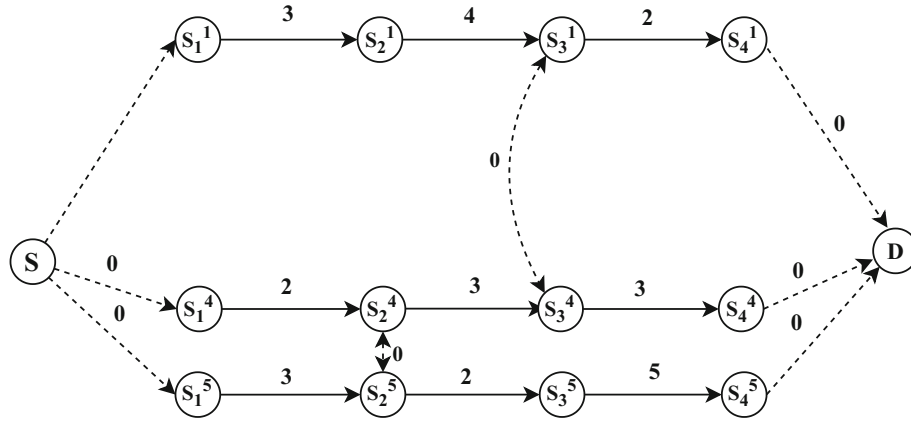


FIGURE 7 | Traversal View of the Network with Imposed Threshold Constraints (see Equations (11) and (12)).

ALGORITHM 1 | Task-Network-Generation.

Input: m as number of workers

Output: Task Network G

1: **procedure** Task_Network_Generate m

2: **for** $i = 1$ to m **do**

3: $\xi_i \leftarrow []$;

4: $\xi_i \leftarrow \xi_i \cup \{(S, S_1^i, w = 0)\}$;

5: $\xi \leftarrow \xi \cup \{(S, S_1^i, w = 0)\}$;

6: $G \leftarrow G \cup \{(S, S_1^i, w = 0)\}$;

7: **for** $j = 1$ to k_i **do**

8: $G \leftarrow G \cup \{(S_j^i, S_{j+1}^i, w = c_j^i)\}$

9: $\xi_i \leftarrow \xi_i \cup \{(S_j^i, S_{j+1}^i, w = c_j^i)\}$

10: $\xi \leftarrow \xi \cup \{(S_j^i, S_{j+1}^i, w = c_j^i)\}$

11: **end for**

12: $G \leftarrow G \cup \{(S_{k_i}^i, D, w = 0)\}$

13: $\xi_i \leftarrow \xi_i \cup \{(S_{k_i}^i, D, w = 0)\}$

14: $\xi \leftarrow \xi \cup \{(S_{k_i}^i, D, w = 0)\}$

15: **end for**

16: **if** $S_j^i \cong S_p^{i \pm d}$ **then**

17: $G \leftarrow G \cup \{(S_p^{i \pm d}, w = 0)\}$

18: **end if**

19: **for** $i = 1$ to m **do**

20: $R[i] \leftarrow \frac{n(\xi_i)}{n(\xi)}$

21: **if** $R[i] < T^\Theta$ **then**

22: **for** $j = 1$ to k_i **do**

23: $\chi \leftarrow \chi \cup \{S_j^i\}$

24: **end for**

25: **end if**

26: **end for**

27: return $(G, \chi, start, destination)$

28: **end procedure**

After finding the ranking for each worker, the condition is checked to see if the rank is below the threshold rank T^Θ . If the condition is true, then all the vertices corresponding to discarding workers are stored in the χ (lines 19–26, Algorithm 1). Now, the defined method function Create_Task_Network() returns χ , DWN G , S , and D (see lines 27, Algorithm 1).

ALGORITHM 2 | Worker-Set-Selection.

Input: m number of the workers

Output: Solution cost of G

1: **procedure** WSC(m)

2: $G, \chi, S, D \leftarrow \text{Task_Network_Generate}(m)$

3: **for each node in** G **do**

4: $Dist[node] \leftarrow \infty$;

5: $Prev[node] \leftarrow \text{null}$

6: **if** $node \neq S$ **then**

7: $Q \leftarrow Q \cup \{S\}$

8: **end if**

9: $Dist[S] \leftarrow 0$

10: **end for**

11: **while** $Q \neq \emptyset$ **do**

12: Mark as visited to all vertices of set χ

13: $V \leftarrow$ node with least $Dist$ in G , mark Q as visited

14: **for** unvisited neighbor N of V **do**

15: $Temp_Dist \leftarrow Dist[V] +$

distance_between(V, N)

16: **if** $Temp_Dist < Dist[N]$ **then**

17: $Dist[N] \leftarrow Temp_Dist$

18: $Prev[N] \leftarrow V$

19: **end if**

20: **end for**

21: **end while**

22: return $Dist, Prev$

23: **end procedure**

Now, the procedure WSC() receives the task network G , χ , S , and D (line 2, Algorithm 2). The values are stored in $Dist$ and $Prev$, priority queue Q except for S (see line 4–10, Algorithm 2). Perform the following actions to remove each node from the queue Q one at a time until the Q is empty:

Designate as visited every node which belongs to χ set. On popping each node V from Q , we iterate each unvisited neighbor N of V and calculate a temporary distance $Temp_Dist$ as the sum of the distance at V and the distance between V and current neighbor N . The distance at the neighbor node N is now updated by the $Temp_Dist$ if it is less than the distance at N . The $Prev$ value is updated with the current node V (see lines 11–21, Algorithm 2). The pushed sequential node to $Prev$ shows the shortest path, and

the function returns the shortest total path cost and shortest path (see line 22 in Algorithm 2).

On imposing the threshold constraint $T^{\theta} = 2$ in the proposed WSC algorithm, the algorithm discards all the solutions of workers who have decomposed the task into two or less than two subtasks only while considering the solution of workers who have decomposed the task more than two subtasks. Accordingly, the workers W_1 , W_4 , and W_5 are deemed feasible candidates to be selected as the winners among six workers W_1 , W_2 , W_3 , W_4 , W_5 , and W_6 with reduced computational effort (see Figures 4 and 7). Now, WSC selects the best-suited workers W_1 and W_4 to solve the task efficiently with minimal effort and resource utilization. The outcome of our objective is a set of selected workers, corresponding subtasks, and an estimated solution cost, represented as the shortest path (see Figure 5). Here, the edges represent the subtasks, the path cost denotes the total solution cost, and the workers selected from the decomposed subtasks.

4.7 | Worker Weight Computation Using Aging Method

We developed a method to update the current information regarding workers' reputation and capacity, enhancing service facilitation on the platform. This method fosters trust among requesters and encourages them to post their tasks on the crowdsourcing platform. The designed aging technique is practically implemented with a pre-set decay rate. To assess its impact on user trust in workers and how it improves the worker's reputation, we conducted a set of experiments at various decay rates. The pseudocode for this method is outlined in Algorithm 3.

The aging technique receives a list of weights W and their corresponding version as an input. First, each weight's age μ is calculated using the version and time (line 3). Next, the weight factor β_F is determined based on the inverse exponential decay rate (line 4). Following this, the normalized weight β_N is computed (line 5). Finally, the desired weight β_R , referred to as the aged weight, is calculated (lines 6–7, Algorithm 3). The time complexity for the aging technique, which calculates the aged weight of workers based on their historical data, is expected to be $\mathcal{O}(n)$.

ALGORITHM 3 | Aging-Weight.

Input: Data-list – weight-list W , Workers' Versions ν , Decay rate λ

Output: ω Aged weight of the worker

1: **function** Aging_technique W, λ

2: **for** $k = 1$ to $W.length$ **do**

3: $\mu[k] \leftarrow \nu[k] - T[k]$

4: $\beta_F[k] \leftarrow e^{-\lambda \cdot \mu[k]}$

5: $\beta_N[k] \leftarrow \frac{\beta_F[k]}{\sum \beta_F[k]}$

6: $\beta_R[k] \leftarrow \beta_F[k] \cdot \beta_N[k]$

7: $\omega \leftarrow \omega + \beta_R[k]$

8: **end for**

9: **return** ω

10: **end function**

4.8 | Best Solution Selection

When requesters submit tasks to the crowdsourcing platform, the platform mechanism maps these tasks to a set of workers for processing. Since requesters have specific constraints and preferences regarding their solutions, whether a solution meets their needs depends on these constraints. For example, if the solution's cost fits in the budget constraints $[B_0, B_1]$ set by the requester, the solution will be accepted. However, if the cost exceeds this range, the requester must either adjust the budget or compromise on the solution's quality.

4.9 | Computing Quality Score of the Alternate Solution Paths

We utilize the aging technique to determine the best solution path among the several available solutions by a set of workers. To demonstrate the concept of working for the quality of the path, we consider the following scenario:

Consider a set of solutions $s_1, s_2, s_3, \dots, s_n$ of a task T posted by a requester to the crowdsourcing platform, and let the $r_1^{s_i}, r_2^{s_i}, r_3^{s_i}, \dots, r_m^{s_i}$ are the decomposed sub-solutions of the solution s_i selected from the m workers, that is, $W_1^{s_i}, W_2^{s_i}, W_3^{s_i}, \dots, W_m^{s_i}$. We assume that there are $m > 2$ number of workers in the collaboration for task T . The workers involved in the solutions of task T have current aged weight $AW_1^{s_i}, AW_2^{s_i}, AW_3^{s_i}, \dots, AW_m^{s_i}$. The definition of the quality score for the solution path s_i is as follows:

$$QW_{s_i} = \sum_{k=1}^{k=m} \left(\frac{r_k^{s_i}}{r_1^{s_i} + r_2^{s_i} + r_3^{s_i} + \dots + r_m^{s_i}} * AW_k^{s_i} \right) \quad (13)$$

4.10 | Budget Window

When requesters submit a task, they specify a budget window within which they are willing to pay for its completion. Our method selects the optimal solution path if multiple solutions are within the requested budget range. To illustrate this concept, we have considered four available solutions, as shown in the table below. We calculate the quality score QW_{p_i} for each solutions $s_1 = 155497df, s_2 = 527eb4c3, s_3 = e2fe75ac, s_4 = fb108f0f$ of τ . The table also details the corresponding information about all solution paths and the set of workers who contributed to them (see Table 4). We discuss three budget window scenarios to demonstrate how the selection of solutions changes when the budget range varies.

Case 0. Budget Window [700–800]

When a requester posts a task on the crowdsourcing platform with a budget range of [700–800], the solution 'fb108f0f' comes under this category, having a quality score of 55.141. Consequently, workers $W_{78}, W_{173}, W_{179}, W_{209}$, and W_{216} are called winners of the task. The crowdsourcing task manager then delivers the task solution to the requester and distributes the cost of the solution among the winning workers.

TABLE 4 | A set of solution paths for a requested task.

Path IDs s_i	Solution cost (\$)	Quality weight	Workers
155497df	830	58.788	$[W_{73}, W_{85}, W_{276}, W_{209}]$
527eb4c3	880	60.407	$[W_{236}, W_{284}, W_{375}, W_{227}]$
e2fe75ac	910	61.479	$[W_{112}, W_{42}, W_{383}, W_{379}, W_{269}, W_{203}, W_{314}]$
fb108f0f	730	55.141	$[W_{209}, W_{179}, W_{78}, W_{173}, W_{216}]$

Case 1. Budget Window [800–900]

In the scenario where the requester selects a budget range of [800–900], two solutions, ‘155497df’ and ‘527eb4c3’, fall within the requested budget. As per our method’s intention to select the highest quality score, the solution ‘527eb4c3’, with a quality weight of 60.407, is chosen as the winner. The workers W_{227} , W_{236} , W_{375} , and W_{284} contributing to current solution are thus declared the winners. It is important to note that in the competitive crowdsourcing environment, a high-quality solution does not necessarily come at a high cost; it depends on the workers’ aged weight and their marginal contribution to the solution within the given limits. The workers receive remuneration for the solution based on their individual contribution percentages.

Case 2. Budget Window [600–700]

Suppose a requester specifies a budget range of [600–700] for a task, and no solution is found. In that case, the user will be notified that no viable solution exists within the specified budget window. In this case, users may need to adjust their budget range or lower the quality requirements to find a feasible solution.

5 | System Simulation And Performance Assessments

This section assesses the effectiveness of the suggested strategy and compares it with existing research. A 12.10 GHz 12th generation Intel(R) Core(TM) i7-12700 CPU with 32.0 GB RAM has been configured to run the proposed model. The environments in which we run the code are VS Code and Jupyter Notebook. To create the directed network, Networkx Python library [38] and plot the proposed work’s performance graph using matplotlib Python library [39].

5.1 | Solution Cost Reduction Experiment

A key objective of crowdsourcing evolution is to encourage skilled workers to participate actively. Requesters seek optimal pricing for their task solutions. At the same time, the platform strives to deliver results that meet their expectations so that they will remain loyal customers for that particular crowdsourcing platform. We are considering the network (Figure 4) [26], on which we are running the existing algorithm, and we find the solution as the shortest path (Figure 5) and the total cost for this solution is 7 unit of cost. The workers involved in this solution are W_1 and W_4 .

Now consider another scenario: the same task is decomposed into four subtasks (Figure 8), again running the same algorithm on

the new directed network, we got the shortest path (Figure 9). Collecting subtasks associated with this shortest path forms the solution for the entire task. Now, the solution cost is 3 units, a reduced cost of increasing the number of subtasks over the previous cost. And the workers who are contributing to this solution are W_2 , W_4 , and W_5 . We examined the statement that increasing the number of subtasks decomposed by the workers reduced the solution cost.

5.2 | WSC Performance Evaluation Experiment on Directed Task Network

The requester posts a decomposable task on the crowdsourcing platform to get the solution. A pool of workers associated with the platform accepts the task, decomposes it into subtasks, and assigns a solution cost to each subtask. The platform then constructs a directed network comprising the subtasks, their associated solution costs, and additional dummy edges. By varying the number of workers, we generate a directed network for each instance and apply the existing and proposed WSC algorithms to evaluate and compare their performance. We have two algorithms to compare their performance:

1. Existing algorithm
2. WSC algorithm

We employ the following three matrices to examine the effectiveness of the suggested method, WSC:

1. Vertices traversal count.
2. Edges traversal count.
3. Algorithms execution time.

On variation in the number of workers:

To evaluate the proposed approach’s performance, we vary the worker pool size and assign a certain number of workers to solve each task at one instance. For both algorithms, we calculate the number of vertices and edges traversed and the execution time. We plot the line chart to compare the proposed algorithm’s performance against the existing algorithm based on the given metrics. We have considered a list of the number of workers; when a decomposable task is submitted, a certain number of workers from that list is assigned to solve it. Each worker in the pool decomposes the received task into a certain number of subtasks according to their convenience. The proposed experiment eliminates a large group of workers who decompose the task into fewer subtasks than a predetermined threshold (T^{θ}) or whose

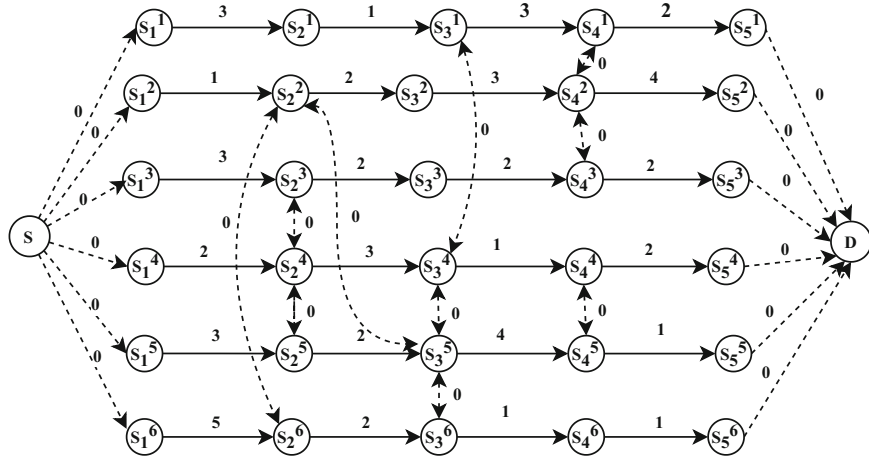


FIGURE 8 | Directed network with an increased number of subtasks.

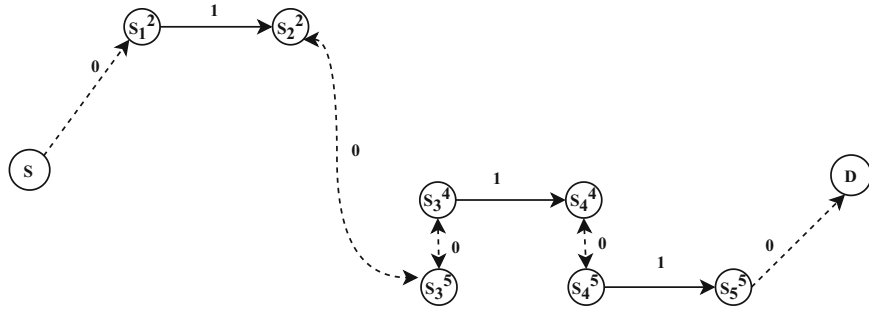


FIGURE 9 | Solution path with an increased number of subtasks.

rank falls below a pre-set threshold rank (R_θ). We set threshold $T^\theta = 5$ (equivalent worker ranking threshold $R_\theta = 4.1$) for all three metrics. We then plot a line chart to illustrate the relationship between the worker pool size and three factors: the number of traversal vertices, the number of traversal edges, and the execution time. In the line chart, the performance of the proposed method is indicated by the green line (triangle-marked), while the performance of the existing method is represented by the red line (circled). On the x-axis, we have considered the number of workers, and on the y-axis, the number of vertices, edges, and execution time.

To analyze the number of vertices traversed by both algorithms, we ran each algorithm on discrete sizes of the worker pool. The number of workers in the pool of the different instances is represented at the x-axis, while the number of vertices traversal by both algorithms is represented at the y-axis (Figure 10). Our findings show that the proposed algorithm requires fewer vertices to be traversed than the existing algorithm to find the shortest path solution. To illustrate this numerically, we look at the pool size of 21 workers; the proposed WSC algorithm must traverse only 97 vertices while the existing algorithm must cover 137 vertices to reach the shortest path results (see Figure 10a).

To analyze the number of edges traversed by both algorithms, we execute both existing and WSC algorithms at different numbers of pool workers working for the crowdsourcing platform. The number of workers in each instance is shown on the x-axis, and the number of edges both algorithms traverse is represented on the

y-axis. (see Figure 10). The green line of the WSC performs better than the red line of the existing algorithm regarding the number of edges metric traversal. To show the performance result, we have taken into account 21 workers. In this instance, the existing algorithm traverses 117 edges while the WSC visits only 77 edges (see Figure 10b).

To assess the execution time performance, we use the same pool of workers to run both algorithms. We count the algorithms' traversal time in milliseconds. The proposed WSC algorithm demonstrates shorter time intervals in each instance of the number of workers than the existing algorithm. For a numerical example, we consider the pool size of worker 21; the existing algorithm needs an execution time of 1.68 milliseconds to process the results of finding the shortest path, while WSC executes the same operation in 1.00 milliseconds (see Figure 10c).

In all three considered metrics—number of nodes, number of edges traversed, and execution time—our proposed algorithm, WSC, performs better than the existing algorithm; implementing the WSC algorithm on a crowdsourcing platform in the real world can provide the solution at the optimal price and efficiently.

5.3 | WSC Performance Evaluation on Randomly Generated Directed Networks

To evaluate the performance of the WSC against the existing method, we set up a separate experimental environment where computer-created directed networks with randomly generated

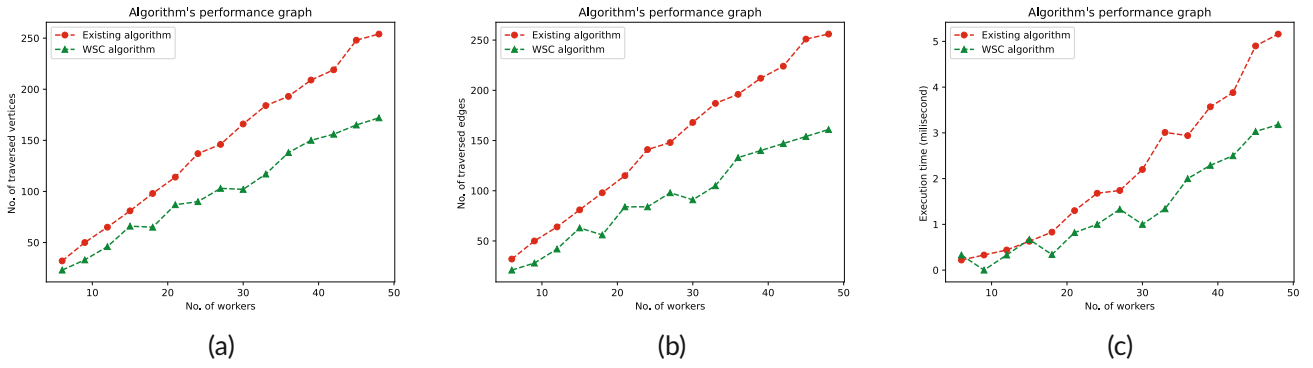


FIGURE 10 | Performance of the Proposed method over existing method, (a) Task graph node traversal performance, (b) Task graph edge traversal performance, and (c) Performance comparison with execution time.

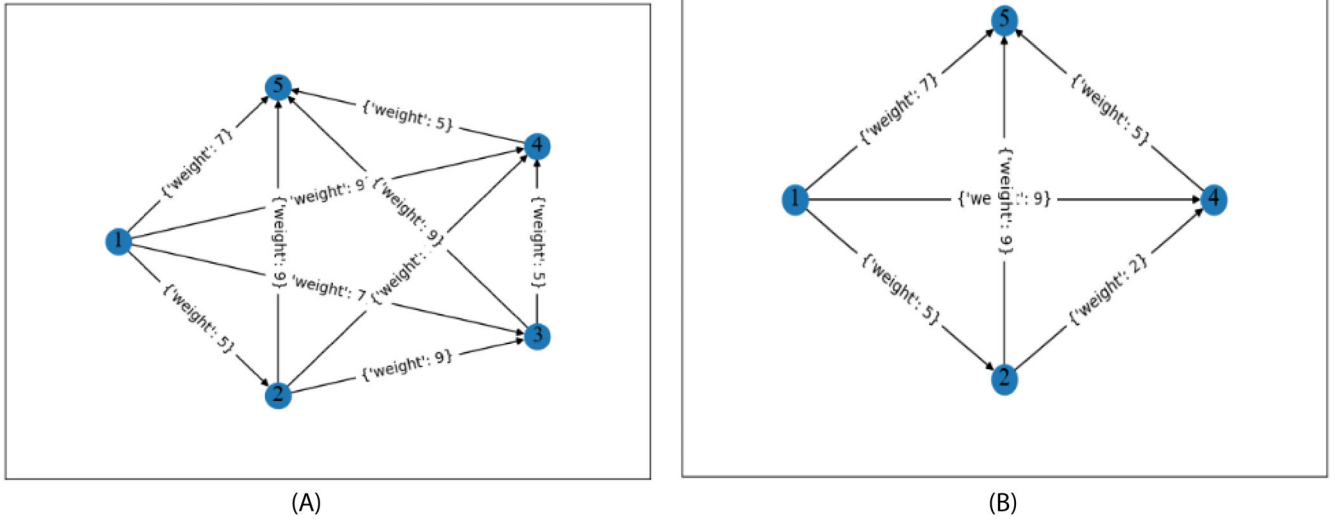


FIGURE 11 | Generated complete graph and feasible solution area, (a) Complete Graph and (b) WSC algorithm traversal view on the complete graph.

edge weights. In this setup, we generated complete graphs of various sizes to serve as the directed networks for testing the algorithms. In one instance, a number is considered, and a complete random graph of that number of nodes is generated. We generate the different complete graphs iterating over different numbers of nodes. A demo of the generated complete graph by the computer with the help of networkx library of 5 nodes is shown (Figure 11a). The weights assigned to the generated graph are taken in the range of (1–10). In this complete graph of 5 nodes, WSC discards node number 3, through which a solution is not feasible; now, we have a view of the traversal remaining subgraph (see Figure 11b). In the same way, the other-directed network is generated by varying the number of nodes. We can use a different policy to discard some nodes (as the threshold does).

We applied both algorithms to this randomly generated directed network. For the proposed algorithm, we implemented a policy to discard certain paths deemed infeasible for solutions. The

performance graphs follow the same convention: the red line represents the existing algorithm performance, while the green line shows the performance of the WSC algorithm. We again consider the three metrics-number of nodes of traversal, number of edges to be traversal, and execution time to compare the results on this randomly generated directed network of both algorithms as listed below:

1. Existing algorithm
2. WSC algorithm

We employ three matrices again to assess the effectiveness of the suggested method, WSC over the existing:

1. Vertices traversal count.
2. Edges traversal count.
3. Algorithms execution time.

Experimental analysis of varying the number of nodes in a complete directed graph:

We consider a list of the number of nodes to generate the complete directed network with each number of nodes of the list. The computer randomly generates the weights of edges for these networks between 1 and 10. Now, this network is ready to be fed into both algorithms. To discard a set of paths or nodes where the feasible solutions are void, WSC uses a specific policy. A complete directed network is constructed, with a source node and an ending node selected. The policy ensures that the source and destination nodes are not included in any discarded paths; otherwise, the algorithm would fail to find the shortest path. The WSC is so intelligent that if the source S or destination D or both are found to be discarded, it ignores them and runs the algorithm. The designed policy that is used in our experiment setup is as follows:

Threshold policy: Consider a complete graph with n nodes, labeled as $n_0, n_1, n_2, \dots, n_k, \dots, n_n$ where n_k signifies k^{th} node of the n -complete directed network. The policy selects an integer randomly $\sigma \in [2, 6]$ and considers that all the nodes satisfying the given equation $n_k \% \sigma = 0$ do not have any feasible solution path. If σ satisfying the equation (if $S \% \sigma == 0$ and/or $D \% \sigma == 0$ is/are true) includes either the source node or destination node or both, then the σ is re-selected until the correct σ is found the correct value.

To illustrate the performance of our proposed WSC algorithm on a randomly generated directed network, we use a line graph to compare it across three key metrics: number of vertices, number of edges traversal, and execution time. The red line (marked circles over the line) depicts the existing algorithm's performance, while the green line (marked triangle over the line) displays the WSC performance.

On the x-axis, we have shown the number of nodes of the completed directed network, and on the y-axis, we indicate the number of nodes traversed during the running of both algorithms (see Figure 12a). The WSC algorithm traverses fewer nodes than the existing one because the proposed algorithm prunes several nodes. If we examine a 45-node example in a generated complete directed network, the existing algorithm traverses 45 vertices. Conversely, our proposed WSC requires 40 vertices to traverse

to find the shortest path. In the performance evaluation of the metric for the number of edges traversed in the complete graph, the x-axis represents the number of vertices in each instance. At the same time, the y-axis shows the number of edges traversed by both algorithms. The red-circled line represents the existing algorithm, while the green-circled line depicts the proposed algorithm. Our analysis demonstrates that the proposed WSC algorithm consistently traverses fewer edges to reach the shortest path than the existing algorithm. If we look at the numerical data of the traversal edge, we consider the instance of 25 nodes of the complete directed network. In this instance, we find the existing algorithm requires visiting 300 edges while the proposed WSC makes traversal only 253 edges (see Figure 12b).

To evaluate the performance of both algorithms based on execution time, we plot the number of nodes in the complete directed network on the x-axis and the execution time on the y-axis in a line graph. Our analysis reveals that the proposed WSC algorithm consistently finds the shortest path with a shorter turnaround time than the existing algorithm. For instance, in a completely directed network with 25 nodes, the existing algorithm takes 13.50 milliseconds to process the results. In contrast, the proposed WSC algorithm completes the task in just 11.01 milliseconds (see Figure 12c). Moreover, there are instances where the execution time of the WSC algorithm is nearly equal to or even slightly higher than that of the existing algorithm, particularly in graphs with a small number of nodes (fewer than 25). This anomaly may be attributed to variations in CPU idleness or busyness during execution. As the number of vertices of the generated complete graph increases, the execution time for existing and WSC algorithms also rises due to the growing network size. Consequently, the computational time scale approximately $O\left(\frac{n*(n-1)}{2}\right) \approx O(n^2)$.

5.4 | Impact of Applying Aging Technique

This section presents the impact of the aging technique on various scenarios involving dynamic weight distribution among workers. We analyze two cases of weight distribution to observe how the aging technique affects worker performance. In the first case, we focus on the workers whose dynamic weights are

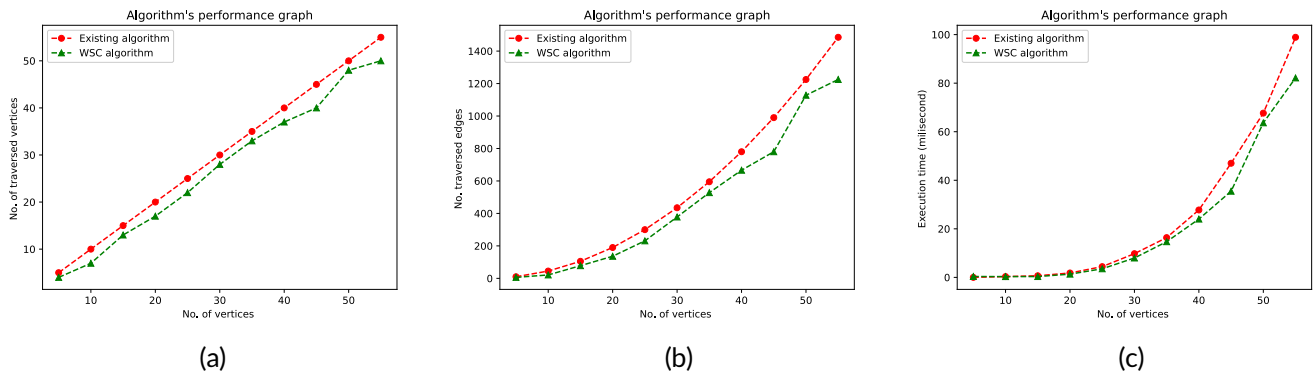


FIGURE 12 | Performance of the Proposed method over existing method, (a) Task graph node traversal performance, (b) Task graph edge traversal performance, and (c) Performance comparison with execution time.

non-decreasing over time and consider all workers in the second scenario whose weights exhibit a non-increasing trend. We have recorded data in a CSV file on the dynamic weights of workers involved in solving a task when submitted to the requester, and feedback is provided based on the user's satisfaction. We utilize the bar graph to illustrate how the aging technique impacts the different cases of dynamic weight. Considering their records, this technique is applied to a list of workers' previous weights. The aging technique calculates the aged weight based on a predetermined decay rate and prior weights. We use symbol λ to represent the decay rate of the aging function throughout the article. The Indian red-colored bar in the bar graph indicates the current weight of that worker. The other colored bar graph shows the workers' aged weights.

The x-axis shows each list of dynamic weights, and the y-axis indicates the worker's dynamic weights and aged weights in a different bar graph. The aged weights at $\lambda = 0.2, 0.4$, and 0.6 are shown in gray, dark gray, and light gray bar graphs. At the $\lambda = 0.3, 0.6$ and 0.9 , the aged weights of the worker were shown in the deep sky blue, sky blue, and light blue bar graphs, respectively.

Case 1. When the dynamic weights of the worker are in non-decreasing order.

The worker W_{352} has a non-decreasing sequence of dynamic weights with a static weight of 61. To demonstrate the performance, we sliced the dynamic weight lists into several lists for indexing on the x-axis. The first list, which includes static weight, is formed $[61, 68.2]$, in which 68.2 is the current dynamic weight of the worker W_{352} . The aged weights of this list are 66.41, 65.18, and 64.31 at $\lambda = 0.2, 0.4$, and 0.6 , respectively. The following index (index 1 at the x-axis) is the dynamic weight list $[61, 68.2, 71.6]$. This dynamic weights list has the aged weights of 68.39, 66.53, and 65.37 at the $\lambda = 0.2, 0.4$, and 0.6 , respectively. At index 2^{nd} dynamic weight list is $[68.2, 71.6, 72.4, 74.4]$, which aged weights are 72.24, 71.19, 70.59 at the $\lambda = 0.2, 0.4$, and 0.6 respectively (see

Figure 13a). Since the list shows the ascending weight, we observe that the aging technique reduced the worker's weight based on the previous records and decay rate in all scenarios. Moreover, as the decay rate increases, the aged weights are reduced. We can see the aging performance in the graph on the following indexes.

We plot the bar graph for the same data non-decreasing dynamic weights, considering the decay rate $\lambda = 0.3, 0.6$, and 0.9 for the aging function (see Figure 13b). Since the workers' dynamic weights are non-decreasing, the aged weight is reduced depending on the decay rate and previous weight distribution during the period. The decay rate is influenced by user involvement duration, task-winning probability, and user satisfaction on the crowdsourcing platform.

Case 2. When the dynamic weights of the worker are in non-increasing sequence.

We select the worker W_{172} having the dynamic weights in non-increasing order, whose static weight is 56. The first list is $[56, 56.8]$ indicated at the 0^{th} index in which the current dynamic weight of the worker W_{172} is 56.8, and 56.60 is the aged weight of the list at the $\lambda = 0.2$. Similarly, at $\lambda = 0.4$ and 0.6 , the aged weights are 56.46 and 56.3, respectively. Since the static weight is below the current dynamic weight, the aging technique reduces the aged weight. The list at the index 1^{st} is $[56, 56.8, 56.0]$; its aged weight at $\lambda = 0.2$ is 56.55, which is more than the current weight. The aged weight on other $\lambda = 0.4$ and 0.6 are 56.41 and 56.33, respectively. Since the dynamic weights of the worker W_{172} are decreasing, we find the aged weights are increased on this non-increasing list to compensate for the previous records. The aged weight also decreases with the increase in decay rate, which adjusts the worker's weight (see Figure 14a). In the same way, the aging performance is shown in 3^{rd} , 4^{th} , and 5^{th} for the other subsequent lists.

Similarly, we have used the dynamic weight data of worker W_{172} to produce the bar graph at the $\lambda = 0.3, 0.6$, and 0.9 represented

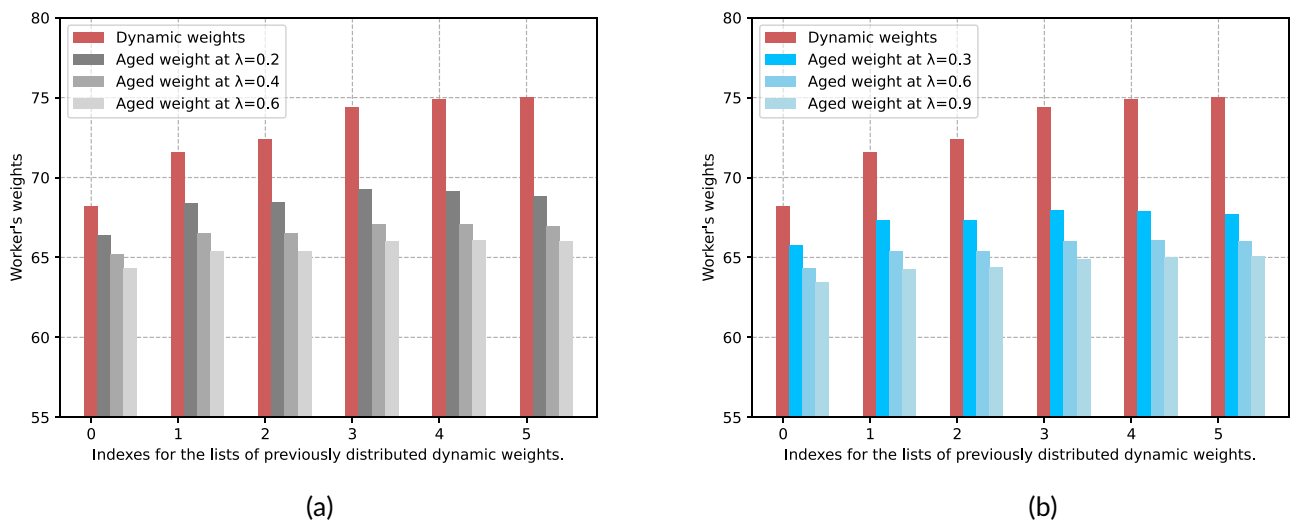


FIGURE 13 | Aging weight variation at different decay rates when workers' performance is increasing, (a) Aged weight performance at different ($\lambda = 0.2, 0.4$, and 0.6) on non-decreasing worker's dynamic weights, (b) Aged weight performance at different ($\lambda = 0.3, 0.6$, and 0.9) on non-decreasing worker's dynamic weights.

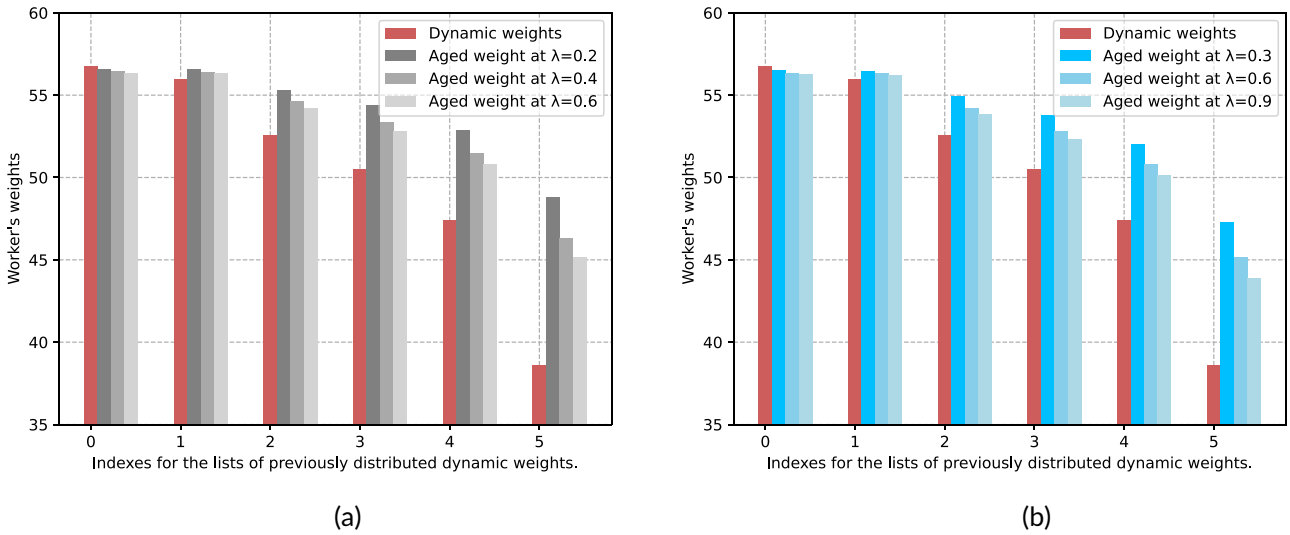


FIGURE 14 | Aging weight variation at different decay rates when workers' performance is decreasing, (a) Aged weight performance at different ($\lambda = 0.2, 0.4$, and 0.6) on non-increasing worker's dynamic weights, (b) Aged weight performance at different ($\lambda = 0.3, 0.6$, and 0.9) on non-increasing worker's dynamic weights.

in the deep blue, sky blue, and light blue. The aged weight of the list at index 0^{th} is 55.52 with a decay rate of 0.3. At the decay rate of 0.6 and 0.9, aged weights are 56.36 and 56.26, respectively. At the dynamic list's 1^{st} index [56.8, 56.0, 52.6], the aged weight at a decay rate of $\lambda = 0.3$ is 55.11. When the decay rate is decreased to $\lambda = 0.6$, the aged weight is reduced to 54.33. This reduction continues as the decay rate increases, with the aged weight at $\lambda = 0.9$ being 53.89. We find that the aged weight increases in this list of dynamic data, which is decreasing (see Figure 14b). We find the same performance at 2^{nd} , 3^{rd} , 4^{th} and 5^{th} index.

6 | Conclusion And Future Work

This paper introduces an in-depth crowdsourcing model that promotes effective collaboration among workers, crowdsourcing platforms, and requesters. We explain how the crowdsourcing platform allows requesters to submit their tasks to the pool of workers. We present a WSC algorithm to create a set of subtasks that complete the entire task with the winning workers. Auction-based crowdsourcing environments are the best suited to deploy our model, where requesters submit the decomposable tasks to be solved within the requested budget. It encourages workers to break down the requested tasks into smaller subtasks. The model provides the task solution in the bounded budget and maintains the solutions' quality. The proposed WSC algorithm greedily selects subtasks with optimal cost while avoiding infeasible solution spaces; the algorithm considers the feasibility of the solutions in that space where there are more subtasks than a certain threshold. As a result, the workers are encouraged to break the task down into more subtasks than the threshold to increase their probability of winning. Consequently, the quality of the solution is increased due to the increasing number of workers participating in solving the problem. Sometimes, the workers cannot solve the entire task, so they choose subtasks according to their interests and skills. In this case, the mechanism does not guarantee that the selected subtasks are equivalent to the

fundamental task. Although the mechanism guarantees an optimal cost solution, it's notable that workers who decompose the task into fewer subtasks below the threshold are unlikely to be selected as the task's winners. The threshold concept encourages the decomposition of the decomposable task into a set of many subtasks, ensuring that the solution is provided at the optimal cost and quality. Based on the budget window concept discussed, the solution is selected.

The aging technique determines the aged weights of workers based on their previous records. The quality of the solution is decided by the aged weights of the workers contributing to the solution (see Equation (13)). The aging technique ties the break when two or more solutions are available in the user's requested budget window. Further, we demonstrate the impact of the aging technique on the workers' previous records (dynamic weights) and to ensure fairness between workers and requesters. We illustrate the performance of our proposed algorithm, WSC, which works more efficiently than the state-of-the-art methods. For tasks that can be divided into smaller tasks, the model is effective. The WSC improves 29% vertices traversal 34% edges traversal and saves the execution time 40.47% for the 21 workers working for the crowdsourcing platform in the weighted directed network. The model operates on the decomposable task, and the worker may solve the decomposed subtasks in parallel or sequentially.

All the pool workers work on their decomposed subtasks parallelly. If a worker has more than one solution (associated with the given complete task), there would be a chance of parallel execution provided it has no external dependency (requiring some other worker to complete the task). The information related to the workers and requesters is stored in the centralized data store, which is very susceptible to theft by rival crowdsourcing platforms or cyber attackers seeking to exploit it for their gain. In the future, we will work on task assignment mechanisms that have external dependencies; we will develop the entire task assignment system and logging system using the smart contract to build

the confidence of the workers and requesters associated with the platform and lead the transparency in the crowdsourcing [40, 41].

Data Availability Statement

The source code of WSC performance is available at <https://github.com/SuneelBHU/CCPE-WSC.git>.

References

1. Y. Zhao and Q. Zhu, "Evaluation on Crowdsourcing Research: Current Status and Future Direction," *Information Systems Frontiers* 16 (2014): 417–434, <https://doi.org/10.1007/s10796-012-9350-4>.
2. K. D. Wang, Z. Chen, and C. E. Wieman, "Can Crowdsourcing Platforms Be Useful for Educational Research?," in *Proceedings of the 14th Learning Analytics and Knowledge Conference, LAK 2024, Kyoto, Japan, March 18–22, 2024* (ACM, 2024), 416–425, <https://doi.org/10.1145/3636555.3636897>.
3. Amazon com I, "Amazon Mechanical Turk," accessed August 1, 2023, <https://www.mturk.com/>.
4. Topcoder, "Your work, completed by our talent|Topcoder," accessed January 1, 2024, <https://www.topcoder.com/>.
5. 759 FTPLA, "Hire Freelancers & Find Freelance Jobs Online|Freelancer," accessed August 1, 2023, <https://www.freelancer.com/>.
6. Inc UG, "Upwork," accessed August 1, 2023, <https://www.upwork.com>.
7. Flightfox, "flightfox," accessed August 1, 2023, <https://www.flightfox.com/>.
8. D. Haas, J. Ansel, L. Gu, and A. Marcus, "Argonaut: Macrotask Crowdsourcing for Complex Data Processing," *Proceedings of the VLDB Endowment* 8, no. 12 (2015): 1642–1653, <http://www.vldb.org/pvldb/vol8/p1642-haas.pdf>.
9. Y. Lin, Y. Jiang, Y. Li, and Y. Zhou, "Privacy-Preserving Batch-Based Task Assignment Over Spatial Crowdsourcing Platforms," *Computer Networks* 241 (2024): 110196, <https://doi.org/10.1016/j.comnet.2024.110196>.
10. Y. Zhao, K. Zheng, Y. Cui, H. Su, F. Zhu, and X. Zhou, "Predictive Task Assignment in Spatial Crowdsourcing: A Data-Driven Approach," in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20–24, 2020* (IEEE, 2020), 13–24, <https://doi.org/10.1109/ICDE48307.2020.00009>.
11. Y. Zhao, K. Zheng, Z. Wang, et al., "Coalition-Based Task Assignment With Priority-Aware Fairness in Spatial Crowdsourcing," *VLDB Journal* 33, no. 1 (2024): 163–184, <https://doi.org/10.1007/s00778-023-00802-3>.
12. R. Esmailyfard, "Improving Detection of Web Service Antipatterns Using Crowdsourcing," *Journal of Supercomputing* 78, no. 5 (2022): 6340–6370, <https://doi.org/10.1007/s11227-021-04134-3>.
13. T. Mahindra, "Tech Mahindra Launches a Global Crowdsourcing Platform," Populii, accessed December 13, 2023, <https://www.techmahindra.com/en-in/tech-mahindra-launches-a-global-crowdsourcing-platform-populii/#:~:text=Tech%20Mahindra%2C%20a%20leading%20provider,in%2Dthe%2Dloop%20services>.
14. T. Mahindra, "Tech Mahindra Launches a Global Crowdsourcing Platform," Populii, accessed December 26, 2023, <https://www.techmahindra.com/>.
15. J. Lee, D. Lee, and S. Kim, "CrowdSky: Skyline Computation With Crowdsourcing," in *Proceedings of the 19th International Conference on Extending Database Technology, EDBT 2016, Bordeaux, France, March 15–16, 2016* (OpenProceedings.org, 2016), 125–136, <https://doi.org/10.5441/002/edbt.2016.14>.
16. Y. Tong, Z. Zhou, Y. Zeng, L. Chen, and C. Shahabi, "Spatial Crowdsourcing: A Survey," *VLDB Journal* 29, no. 1 (2020): 217–250, <https://doi.org/10.1007/s00778-019-00568-7>.
17. J. Fan, G. Li, B. C. Ooi, K. Tan, and J. Feng, "iCrowd: An Adaptive Crowdsourcing Framework," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31–June 4, 2015* (ACM, 2015), 1015–1030, <https://doi.org/10.1145/2723372.2750550>.
18. S. Gao, X. Chen, J. Zhu, X. Dong, and J. Ma, "TrustWorker: A Trustworthy and Privacy-Preserving Worker Selection Scheme for Blockchain-Based Crowdsensing," *IEEE Transactions on Services Computing* 15, no. 6 (2022): 3577–3590, <https://doi.org/10.1109/TSC.2021.3103938>.
19. X. Li, Y. Wang, B. Xie, et al., "Planning-Based Mobile Crowdsourcing Bidirectional Multi-Stage Online Task Assignment," *Computer Networks* 233 (2023): 109879, <https://doi.org/10.1016/j.comnet.2023.109879>.
20. L. Zheng, L. Chen, and P. Cheng, "Privacy-Preserving Worker Allocation in Crowdsourcing," *VLDB Journal* 31, no. 4 (2022): 733–751, <https://doi.org/10.1007/s00778-021-00713-1>.
21. Y. Tong, L. Chen, and C. Shahabi, "Spatial Crowdsourcing: Challenges, Techniques, and Applications," *Proceedings of the VLDB Endowment* 10, no. 12 (2017): 1988–1991, <http://www.vldb.org/pvldb/vol10/p1988-tong.pdf>.
22. P. Zhao, G. Zhang, S. Wan, G. Liu, and T. Umer, "A Survey of Local Differential Privacy for Securing Internet of Vehicles," *Journal of Supercomputing* 76, no. 11 (2020): 8391–8412, <https://doi.org/10.1007/s11227-019-03104-0>.
23. S. Han, Z. Xu, Y. Zeng, and L. Chen, "Fluid: A Blockchain Based Framework for Crowdsourcing," in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30–July 5, 2019* (ACM, 2019), 1921–1924, <https://doi.org/10.1145/3299869.3320238>.
24. Y. Liang, Y. Li, and B. Shin, "Decentralized Crowdsourcing for Human Intelligence Tasks With Efficient On-Chain Cost," *Proceedings of the VLDB Endowment* 15, no. 9 (2022): 1875–1888, <https://www.vldb.org/pvldb/vol15/p1875-shin.pdf>.
25. E. W. Dijkstra, "A Note on Two Problems in Connexion With Graphs," *Numerische Mathematik* 1 (1959): 269–271, <https://doi.org/10.1007/BF01386390>.
26. S. K. Mridha and M. Bhattacharyya, "A Network Based Mechanism for Managing Decomposable Tasks via Crowdsourcing," *Electronic Commerce Research* 18, no. 4 (2018): 869–881, <https://doi.org/10.1007/s10660-018-9317-8>.
27. EC of India, "Disclosure of Electoral Bonds," accessed June 10, 2024, <https://www.eci.gov.in/disclosure-of-electoral-bonds>.
28. N. Pirttinen, "Crowdsourcing in Computer Science Education," in *ICER 2021: ACM Conference on International Computing Education Research, Virtual Event, USA, August 16–19, 2021* (ACM, 2021), 421–422, <https://doi.org/10.1145/3446871.3469781>.
29. S. K. Mridha and M. Bhattacharyya, "Network Based Mechanisms for Competitive Crowdsourcing," in *Proceedings of the ACM India Joint International Conference on Data Science and Management of Data, COMAD/CODS 2018, Goa, India, January 11–13, 2018* (ACM, 2018), 318–321, <https://doi.org/10.1145/3152494.3167979>.
30. S. K. Mridha, "Appropriate Co-Worker Recommendation in Crowdsourcing," in *CODS-COMAD 2021: 8th ACM IKDD CODS and 26th COMAD, Virtual Event, Bangalore, India, January 2–4, 2021* (ACM, 2021), 425, <https://doi.org/10.1145/3430984.3431052>.
31. J. Kim, S. Sterman, A. A. B. Cohen, and M. S. Bernstein, "Mechanical Novel: Crowdsourcing Complex Work Through Reflection and Revision," in *Proceedings of the 2017 ACM Conference on Computer Supported Cooperative Work and Social Computing, CSCW 2017, Portland,*

OR, USA, February 25–March 1, 2017 (ACM, 2017), 233–245, <https://doi.org/10.1145/2998181.2998196>.

32. G. Goel, A. Nikzad, and A. Singla, “Mechanism Design for Crowdsourcing Markets With Heterogeneous Tasks,” in *Proceedings of the Second AAAI Conference on Human Computation and Crowdsourcing, HCOMP 2014, November 2–4, 2014, Pittsburgh, Pennsylvania, USA* (AAAI, 2014), 77–86, <https://doi.org/10.1609/hcomp.v2i1.13158>.

33. L. Gao, T. Cheng, and L. Gao, “TSWCrowd: A Decentralized Task-Select-Worker Framework on Blockchain for Spatial Crowdsourcing,” *IEEE Access* 8 (2020): 220682–220691, <https://doi.org/10.1109/ACCESS.2020.3043040>.

34. D. Yang, G. Xue, X. Fang, and J. Tang, “Crowdsourcing to Smartphones: Incentive Mechanism Design for Mobile Phone Sensing,” in *The 18th Annual International Conference on Mobile Computing and Networking, Mobicom’12, Istanbul, Turkey, August 22–26, 2012* (ACM, 2012), 173–184, <https://doi.org/10.1145/2348543.2348567>.

35. M. Kadadha, H. Otrouk, S. Singh, R. Mizouni, and A. Ouali, “Two-Sided Preferences Task Matching Mechanisms for Blockchain-Based Crowdsourcing,” *Journal of Network and Computer Applications* 191 (2021): 103155.

36. B. Satzger, H. Psailer, D. Schall, and S. Dustdar, “Auction-Based Crowdsourcing Supporting Skill Management,” *Information Systems* 38, no. 4 (2013): 547–560, <https://doi.org/10.1016/j.is.2012.09.003>.

37. Y. Gao and A. G. Parameswaran, “Finish Them!: Pricing Algorithms for Human Computation,” *Proceedings of the VLDB Endowment* 7, no. 14 (2014): 1965–1976, <http://www.vldb.org/pvldb/vol7/p1965-gao.pdf>.

38. Developers N, “NetworkX,” accessed August 1, 2023, <https://networkx.org/>.

39. Development Team TM, “Matplotlib,” accessed February 10, 2023, <https://matplotlib.org/>.

40. T. Xiong, S. Feng, M. Pan, and Y. Yu, “Smart Contract Generation for Inter-Organizational Process Collaboration,” *CoRR* (2023): abs/2303.09257, <https://doi.org/10.48550/arXiv.2303.09257>.

41. K. Kant, S. Pandey, and U. Shanker, “A Journey From Commit Processing in Distributed Databases to Consensus in Blockchain,” in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9–12, 2022* (IEEE, 2022), 3236–3240, <https://doi.org/10.1109/ICDE53745.2022.00306>.